UNIFIED ENGINEERING WORKBENCH

Technical Report | Pass Science Works R&D Internship 2026–2027

Scope: A desktop-oriented engineering workbench combining a C++17 computational backend with a React/TypeScript interface for thermal finite-element analysis, deterministic parallel benchmarking, hardware-adapter simulation, and manually implemented neural-network training.

1. Executive Summary

The workbench presents several engineering workflows in one interface. The supplied application screenshots show a 1D steady-state thermal FEM solver with analytical comparison and mesh-convergence results, a serial-versus-parallel benchmark, and a small regression network trained using handwritten backpropagation with a central finite-difference gradient check.

This report transcribes the values visible in the screenshots; it does not independently reproduce every experiment. The native Windows 11 clean-machine launch requirement remains a separate acceptance test and must not be described as verified unless it has actually been run on Windows.

2. System Architecture

Layer	Role	Technology / evidence
Presentation	Navigation, parameter entry, plots, result tables, status and CSV export.	React, TypeScript, Vite; geometry and charts are presented in the UI.
Service interface	Connects UI workflows to computational operations and reports status/errors.	HTTP backend; screenshot shows backend connected at localhost:8080.
Numerical backend	Thermal FEM, benchmark workload, and neural-network calculations.	C++17 implementation as described in project materials.
Hardware adapters	Represent device state/telemetry and connection failures without physical hardware.	Mock/simulated adapters; physical hardware integration is not claimed.

Design boundary: This report describes the demonstrated 1D thermal solver and visible application workflows. It does not claim validated 2D/3D finite-element capability or real hardware communication.

3. Thermal Finite-Element Analysis

The thermal module solves the one-dimensional steady-state conduction problem with uniform internal heat generation:

$$-d/dx(kA\,dT/dx) = qA, \text{ with } T(0) = 100\ °C \text{ and } T(L) = 20\ °C.$$

Displayed nominal parameters: L = 1 m, A = 1.0 × 10^{-4} m^2, k = 10 W/(m·K), and q = 10,000 W/m^3. The analytical solution used for comparison is:

$$T(x) = 100 - 80x/L + [q/(2k)]x(L - x).$$

For linear two-node elements, the element stiffness matrix and consistent source vector are:

$$K_e = (kA/h)\,[[1, -1], [-1, 1]];\ F_e = (qAh/2)\,[1, 1]^T.$$

3.1 Mesh convergence results

Elements	h (m)	Max midpoint error (°C)	Error ratio	Displayed status
10	0.1000	1.250000	—	O(h²) Quadratic
20	0.0500	0.312500	4.00	O(h²) Quadratic
40	0.0250	0.078125	4.00	O(h²) Quadratic
80	0.0125	0.019531	4.00	O(h²) Quadratic

The reported error falls by approximately a factor of four whenever the element size is halved, consistent with second-order error behaviour for this linear-element problem. The screenshot shows a 20-element midpoint error of 0.3125 °C.

Interpretation: the FEM curve closely overlays the analytical curve at the displayed scale, and the convergence table provides a quantitative mesh-refinement check. Retain the exported result CSV/log alongside the report if raw evidence is required.

Evidence: Thermal Analysis screenshots supplied by the candidate; values transcribed from the visible convergence table and status badge.

4. Parallel Performance Benchmark

The Performance Lab screen reports a deterministic benchmark with seed 42, one warm-up, five measured repetitions, 50 independent problem cases, and 40 mesh elements per case. The displayed environment is arm64 (Darwin), 10 logical cores, Clang 21.0.0, optimisation flag −O2.

Workers	Median time (ms)	Displayed speedup	Displayed efficiency
1 (serial)	1.40	1.00×	100%
2	0.62	2.25×	112%
3	0.43	3.29×	110%
4	0.36	3.91×	98%

The UI reports peak speedup of 3.91× at four workers and numerical agreement of 0.00e+0 °C against a stated threshold of 1e−8 °C. The individual repetition values vary; for example, the four-worker run includes a 0.51 ms repetition, so median-based reporting is appropriate.

The displayed efficiency above 100% for two and three workers can occur in a small benchmark due to timing noise, scheduling, and cache effects; it should not be interpreted as physical efficiency greater than 100%. The sub-2 ms runtime may also be sensitive to timer resolution and system load.

These values are for the displayed Apple arm64/Darwin environment and do not establish equivalent performance on Windows. For reproducibility, include the exported benchmark CSV and identify the exact executable/build used.

Evidence: Performance Lab screenshot supplied by the candidate.

5. Neural Network and Gradient Verification

The Neural Network Lab screen shows a single-sample regression model trained for 100 steps at learning rate $\eta = 0.1$, with target $x = 1.0$ and $y = 2.0$. The displayed model is:

$$h = \tanh(wx + b);\ \hat{y} = vh + c;\ J = \tfrac{1}{2}(\hat{y} - y)^2.$$

Initial parameters are $w = 0.5$, $b = 0.0$, $v = 1.0$, and $c = 0.0$. The gradient-check panel reports maximum central finite-difference discrepancy 1.13×10^{-10}, using $\varepsilon = 10^{-6}$, against a stated tolerance of 1×10^{-5}.

For each parameter θ, the numerical derivative is estimated by:

$$\partial J/\partial \theta \approx [J(\theta + \varepsilon) - J(\theta - \varepsilon)] / (2\varepsilon).$$

The training-loss plot reports final loss 2.6830×10^{-20} after 100 iterations. This is a compact correctness demonstration, not evidence of generalisation on a dataset. Retain the exported training-history CSV if available.

Evidence: Neural Network Lab screenshot supplied by the candidate.

6. Geometry and Data Visualisation

Project materials identify a geometry workflow for importing mesh geometry and temperature CSV data, with visualisation in the workbench UI. Example data files are described as cube/cylinder temperature test CSVs and connected-cylinder mesh files. Before submission, verify that the packaged application loads at least one provided ASCII STL and a CSV matching the required schema, and retain a screenshot showing the imported model and mapped temperature field.

State data provenance in the interface or report: source filename, units, coordinate system, and whether values are measured, synthetic, or generated for testing. Do not label synthetic test values as experimental measurements.

7. Embedded Hardware Mock Design

The Hardware view is intended to demonstrate software interfaces for MCU, FPGA, and debug-adapter workflows without requiring physical equipment. The project documentation describes simulated device state and telemetry, including connection/fault handling. These are treated as mocks rather than validated electrical interfaces.

Adapter category	Purpose	Evidence to include
MCU mock	Represent target status and telemetry.	Device panel screenshot; connection/disconnection log.
FPGA mock	Represent FPGA target state and interface signals.	Target/interface diagram with voltage domains labelled.
JTAG/debug mock	Represent debug connection, scan state, and failure recovery.	Pin mapping, voltage assumptions, reconnect/failure screenshot.

Check any pinout and voltage values against the exact target-board and adapter datasheets. A conceptual diagram is not a substitute for electrical validation.

8. Testing, Reproducibility, and Limitations

The project notes report a C++ test suite and UI-level workflows, but this report does not independently execute the source tree. State test counts only when backed by the actual test log. Attach CTest output, frontend type-check/build output, and benchmark/training CSVs as evidence; avoid relying on UI badges alone.

Known scope limits

- Thermal solver: demonstrated as a one-dimensional rod problem; no validated 2D/3D continuum FEA is claimed.
- Geometry: verify supported STL variants; binary STL should not be claimed unless tested.
- Hardware: mock adapters only unless physical hardware tests were performed.
- Performance: timings are machine- and build-specific; shown values are from arm64 Darwin.
- Deployment: screenshots do not prove clean Windows 11 x64 Launch.exe behaviour.

9. Submission Acceptance Checklist

Item	Status / action
Technical report	Structured, screenshot-backed summary; add actual raw-log/CSV references if required.
Walkthrough PDF	Include the separate walkthrough/log PDF; ensure screenshots are genuine and readable.
Handwritten derivation appendix	Include scanned handwritten pages; check legibility and page limit.
AI disclosure and ownership	Include project files and verify statements accurately reflect assistance and authorship.
Demo evidence	Screenshots can replace video only if the assignment permits it; include logs and clear step-by-step captions.
Git history / bundle	Verify the bundle opens and contains the intended commits.
Final ZIP / checksum	Rebuild the ZIP after final edits, then regenerate the checksum.
Windows launch test	Still requires confirmation on Windows 11 x64 under the specified clean-machine conditions.

Final note

This report is based on the provided UI screenshots and project-context descriptions. It should be submitted alongside the genuine exported results, test logs, source/build instructions, and other required artifacts. Do not present unverified items as tested.

Evidence sources: candidate-supplied Thermal Analysis, Performance Lab, and Neural Network Lab screenshots. Numeric results above are transcribed from those screenshots; no independent execution is implied.